- Dropout randomly disables a fraction of neurons during training, helping to prevent overfitting and improve generalization.

Implementation Example: Image Classification

The following example demonstrates how to build a simple CNN for classifying images from the CIFAR10 dataset.

Step 1: Import Libraries

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
```

Step 2: Load and Preprocess Data

```
# Load dataset
(X_train, y_train), (X_test, y_test) = cifar10.load_data() # Normalize pixel values to the range [0, 1]
X_train, X_test = X_train /255.0, X_test /255.0

# Convert labels to one-hot encoding
y_train, y_test = to_categorical(y_train), to_categorical(y_test)
```

Step 3: Define the Model

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)), # First convolutional layer MaxPooling2D((2, 2)), # First pooling layer Conv2D(64, (3, 3), activation='relu'), # Second convolutional layer MaxPooling2D((2, 2)), # Second pooling layer Flatten(), # Flatten the feature maps Dense(64, activation='relu'), # Fully connected layer Dense(10, activation='softmax') # Output layer for 10 classes ])
```

Step 4: Compile the Model

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

- **Optimizer**: The Adam optimizer combines the benefits of AdaGrad and RMSProp, making it highly effective for CNNs.
- **Loss Function**: Categorical crossentropy is used for multi-class classification tasks.
- **Metrics**: Accuracy is a common metric for evaluating classification performance.

Step 5: Train the Model

```
history = model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
```